Speculative Decoding for Efficient LLM Inference

Anonymous DLCC submission

Paper ID 0001

Abstract

Speculative decoding accelerates large language model inference by using a smaller draft model to propose token blocks that a target model verifies in parallel. Despite a substantial recent literature, reported speedups vary widely with hardware, model pairing, draft length, and sampling regime, and most published numbers come from data-center GPUs. We present a controlled empirical study on consumer hardware (NVIDIA RTX 5090, `int8` target via `bitsandbytes`) using Qwen2.5-7B-Instruct as the target and Qwen2.5-0.5B/1.5B as same-family drafts, evaluated on GSM8K, MMLU subsets, and CNN/DailyMail with frozen sample manifests. We measure throughput, time-to-first-token, time-per-output-token, token acceptance dynamics, and task quality across draft lengths $k \in \{4, 8, 16\}$ and two sampling regimes. The best autoregressive configuration delivers a wall-clock speedup of $1.65\times$ at $k{=}8$ ($\bar{\alpha}{=}0.40$, $\bar{B}_{eff}{=}3.20$), with quality changes bounded within 1.0 point on GSM8K and MMLU. The empirical effective block size tracks the geometric upper bound $(1-\alpha^{k+1})/(1-\alpha)$ within 3%, isolating the linear-in-k draft cost as the binding term once per-block overhead is amortised at $k{\geq}8$. Building on this analysis, we propose DriftDiffuse*, a $\sim$30 M-parameter bidirectional masked-diffusion drafter with a position-conditional drift schedule that proposes k tokens in $n_{denoise} \ll k$ parallel steps while remaining compatible with the standard verifier. We provide a fully reproducible protocol and operating guidelines for selecting draft model size and draft length under explicit quality constraints.*

1. Introduction

Autoregressive decoding in large language models (LLMs) is fundamentally sequential: each output token requires a full forward pass through the target model, capping single-stream throughput at the per-token kernel latency. For 7 B-class models served on a single consumer GPU, this is the dominant cost in latency-sensitive applications such as code completion, on-device assistants, and interactive chat. *Speculative decoding* addresses this bottleneck by interposing a small *draft* model that proposes k candidate tokens, which the target then verifies in parallel with a single forward pass; rejected tokens are re-sampled from the target's residual distribution to preserve the output distribution [10, 4].

The headline numbers for speculative decoding ($S \in [2\times, 5\times]$) are typically reported on data-center GPUs (A100, H100, H200) using unquantised drafters. On consumer hardware with quantised models the picture is murkier: per-block bookkeeping overhead, dequantisation cost, and asymmetric draft/target throughput can erase the algorithmic gain entirely. Practitioners deploying speculative decoding on a single RTX 4090- or 5090-class GPU therefore lack quantitative guidance on which (draft size, k, regime) to pick, and at what point the linear-in-k drafting cost itself becomes the bottleneck.

1.1. Research questions

We address three research questions:

RQ1. How much end-to-end speedup does vanilla speculative decoding deliver against autoregressive decoding on a single RTX 5090 with an `int8` 7 B target?

RQ2. How do draft model size and draft length jointly determine token acceptance and net latency?

RQ3. How sensitive is the speed–quality trade-off to the sampling regime (deterministic vs. stochastic) and to task entropy?

1.2. Contributions

This paper makes the following contributions:

1. A reproducible empirical protocol with frozen sample manifests on three benchmarks (GSM8K, MMLU subsets, CNN/DailyMail) and a 14-configuration grid (Qwen2.5-7B target; 0.5B/1.5B drafts; $k \in \{4, 8, 16\}$; deterministic and stochastic regimes).
2. Wall-clock and acceptance results showing a best speedup of $S{=}1.65\times$ at $k{=}8$ deterministic, and an empirical $\bar{B}_{\text{eff}}$ that tracks the geometric upper bound

within 3%, identifying the linear-in-k draft cost as the binding term once per-block overhead is amortised.

3. *DriftDiffuse*: a $\sim$30 M-parameter bidirectional masked-diffusion drafter with a position-conditional drift schedule and MaskGIT-style iterative unmasking [3], supporting both block-level and Leviathan-style token-level acceptance and producing k draft tokens in $n_{\text{denoise}} \ll k$ parallel steps.
4. A break-even analysis that explains why parallel drafters become attractive at $k \geq 8$ on consumer hardware, together with the complete training and evaluation pipeline distilled from the target's own outputs.

The rest of the paper is organised as follows. Section 2 reviews related work. Section 3 defines the experimental protocol; Section 4 the metrics. Section 5 covers implementation. Section 6 presents the autoregressive results, and Section 7 introduces DriftDiffuse. Section 8 discusses systems implications, and Section 10 concludes.

2. Related Work

Vanilla speculative decoding. Leviathan *et al.* [10] introduced the canonical draft–verify scheme: a smaller *draft* model proposes k tokens autoregressively and the larger *target* verifies them in a single parallel forward pass, with modified rejection sampling guaranteeing that the output distribution is identical to that of the target. Chen *et al.* [4] concurrently proposed an equivalent scheme. The expected per-cycle latency is

$$L = \frac{T_{\text{draft}} + T_{\text{verify}}}{\tau}, \qquad T_{\text{draft}} = k \cdot t_{\text{draft step}}, \tag{1}$$

where $\tau \in [1, k{+}1]$ is the expected number of accepted tokens per cycle. Wall-clock speedup $S = L_{\text{target}}/L$ therefore depends on the draft/target cost ratio and the empirical token acceptance rate α. Stern *et al.* [18] earlier explored blockwise parallel decoding within a single model.

Reducing drafting cost. Medusa [2] eliminates the external draft by attaching multiple prediction heads to the target model and verifying the resulting candidate tree in one forward pass. The EAGLE family [13, 12, 14] keeps an external draft but conditions it on the target's hidden features, reaching $\tau \approx 3$ and $S \approx 2\times$. Despite these refinements, T_{draft} remains linear in k, which forces EAGLE-3 to a single transformer layer and caps achievable speedups at $\sim$2–3$\times$ on strong GPUs.

Diffusion-based drafters. Recent work attacks the linear-in-k drafting bottleneck directly. DiffuSpec [11] and SpecDiff-2 [17] employ pre-trained diffusion LLMs as parallel drafters but rely on $\sim$7 B drafters whose memory footprint and per-step latency are prohibitive in single-stream serving. PARD [1] trains small autoregressive models to mimic diffusion-style parallel generation but caps at $\sim$3$\times$. DFlash [5] closes this gap with a lightweight (5-layer) *block-diffusion* drafter conditioned on target hidden features, reporting up to 5$\times$ end-to-end speedup over autoregressive baselines on H200/B200 hardware.

Position of this work. Our study is the first, to our knowledge, to combine two contributions in the same protocol: (i) a controlled empirical evaluation of vanilla speculative decoding on consumer-class hardware (RTX 5090, Qwen2.5-7B-Instruct quantised to 8-bit via `bitsandbytes` [7]); and (ii) DriftDiffuse, a small ($\sim$30 M parameters) bidirectional masked-diffusion drafter that operates within the consumer-hardware budget while breaking the linear-in-k drafting cost. We do not compete with DFlash's H200-class numbers; we instead establish the operating regime in which a parallel drafter remains useful when the AR draft is already well-tuned.

3. Experimental protocol

3.1. Datasets and sample sizes

We evaluate on three benchmarks spanning reasoning, knowledge, and summarization:

1. **GSM8K** [6] test subset: 300 samples.
2. **MMLU** [8] subset: 500 samples (5 subjects $\times$ 100 each).
3. **CNN/DailyMail** [15] subset: 200 samples.

Sampling policy: Fixed random seed $= 42$, stratified sampling where applicable, and a frozen sample-ID manifest generated before experimentation.

3.2. Prompt set

We use the following fixed prompt templates for each task:

GSM8K:

```
You are a careful math assistant.
Solve the problem step by step and
end with Final Answer: <number>.
Question: {question}
```

MMLU:

```
You are a knowledgeable assistant.
Choose exactly one option.
Question: {question}
Options: A. {A} B. {B} C. {C} D. {D}
Answer:
```

CNN/DailyMail:

```
Summarize the following article in
3--4 sentences, preserving key facts
and entities.
Article: {article}
Summary:
```

3.3. Model setup

- **Target model:** Qwen/Qwen2.5-7B-Instruct (7B).
- **AR draft models** (same family as target):
  (a) Qwen/Qwen2.5-0.5B-Instruct (0.5B; fast draft; lower expected acceptance).
  (b) Qwen/Qwen2.5-1.5B-Instruct (1.5B; balanced draft; expected best trade-off).
- **Tokenizer rule:** Target and draft must use the same tokenizer family.
- **Baseline:** Target-only autoregressive decoding.

3.4. Decoding regimes

- **Deterministic regime:** temperature $= 0.0$, top_p $= 1.0$.
- **Stochastic regime:** temperature $= 0.7$, top_p $= 0.9$.
- **Draft lengths:** $k \in \{4, 8, 16\}$.
- **Max new tokens:** GSM8K: 256; MMLU: 32; CNN/DailyMail: 160.

3.5. Controlled variables

1. Identical prompts and sample order across all runs.
2. Identical hardware and software environment.
3. Fixed seeds for determinism and robustness checks.

4. Evaluation metrics

Table 1 defines the exact reporting metrics used in this study.

4.1. Success criteria

Primary:

1. $S \geq 1.3\times$.
2. Absolute quality drop $\leq$ 1.0 point on GSM8K and MMLU in deterministic regime.

Secondary: Identify best (draft size, k) maximizing S while satisfying the quality constraint.

5. Implementation and Reproducibility

5.1. Code organisation

The pipeline is organised under `src/` as `config.py` (model IDs, prompts, paths), `data_loader.py` (deterministic subsampling under a fixed seed), `baseline.py` (target-only autoregressive loop), `speculative.py` (vanilla draft–verify loop following Leviathan *et al.* [10]), `drift_speculative.py` (DriftDiffuse drafter; Section 7), `metrics.py`, `evaluate.py`, and `visualize.py`. All measurement helpers (CUDA event-based latency, TTFT/TPOT decomposition, seed initialisation, CSV logging) are isolated in `utils.py`.

5.2. Models and quantisation

The target is Qwen2.5-7B-Instruct [16] loaded in 8-bit via `bitsandbytes` [7] with `device_map="auto"`. Drafts are Qwen2.5-0.5B and Qwen2.5-1.5B in `fp16`. We use Hugging Face Transformers' `generate(assistant_model=...)` interface for vanilla speculative decoding [9] to ensure that our acceptance numbers are reproducible from the same library version reported in `env_spec.txt`.

5.3. Sample manifests and seeds

For each dataset we draw a fixed sample-ID manifest with `seed=42` and persist it to `manifests/{gsm8k,mmlu,xsum}.json`; every configuration is evaluated on exactly the same prompt set. For deterministic runs we set `do_sample=False`, `temperature=0`, `top_p=1.0`; for stochastic runs we use `temperature=0.7`, `top_p=0.9`. The two best configurations are re-run with seeds $\{42, 123, 999\}$ to obtain the speedup standard deviation σ_S reported in Section 6.

5.4. Measurement protocol

Latency is measured with `torch.cuda.Event` pairs around the `generate` call, with one warmup sample per configuration to absorb CUDA graph compilation. Per-sample we record `sample_id`, `task`, `latency`, `ttft`, `tpot`, `n_tokens`, `output_text` plus the speculative-only fields `n_draft_calls`, `n_target_calls`, `n_accepted`, `n_proposed`, from which $\bar{\alpha}$ and $\bar{B}_{\text{eff}}$ are derived directly without distributional assumptions.

5.5. Artefact release

The complete code, manifests, raw `results/*.csv`, and the DriftDiffuse training notebook are released under the Assignment 2 repository. The exact CUDA toolkit, driver, Python, and library versions are pinned in `env_spec.txt` alongside this manuscript.

Table 1. Exact metric definitions for reporting.

Metric	Symbol	Computation	Unit	Direction
End-to-end latency	T	completion_time − request_start	s	Lower
Throughput	R_{tok}	output_tokens / T	tokens/s	Higher
Time-to-first-token	TTFT	first_token_time − request_start	ms	Lower
Time-per-output-token	TPOT	(completion_time − first_token_time) / output_tokens	ms/token	Lower
Acceptance rate	α	accepted_draft_tokens / proposed_draft_tokens	ratio	Higher
Effective block size	B_{eff}	accepted_draft_tokens / verify_steps	tokens/step	Higher
Relative speedup	S	baseline_latency / speculative_latency	×	Higher
GSM8K quality	Q_{gsm}	correct / total	%	Higher
MMLU quality	Q_{mmlu}	correct / total	%	Higher
CNN/DailyMail quality	Q_{sum}	standard ROUGE-L F1	score	Higher
Quality delta	ΔQ	$Q_{\text{spec}} - Q_{\text{base}}$	points	≈ 0 or $+$
Speedup stability	σ_S	std(S over seeds)	×	Lower

Table 2. Headline results across the AR speculative grid. $\bar{R}$ is mean throughput (tok/s); $\bar{\alpha}$ is the token-level acceptance rate; $\bar{B}_{\text{eff}}$ is the average accepted-token block length; S is the paired throughput speedup over the matching baseline regime. Best S per regime in bold.

Config	k	Reg.	$\bar{R}$	$\bar{\alpha}$	S
baseline 7B	–	det	9.19	–	1.00
spec 0.5B	4	det	7.10	0.45	0.77
spec 0.5B	8	det	15.19	0.40	**1.65**
spec 0.5B	16	det	14.84	0.30	1.61
spec 1.5B	4	det	14.53	0.44	1.58
baseline 7B	–	stoch	9.12	–	1.00
spec 0.5B	4	stoch	8.51	0.43	0.93
spec 0.5B	8	stoch	13.52	0.39	1.48
spec 0.5B	16	stoch	14.50	0.30	**1.59**

6. Results

This section reports the measured wall-clock and acceptance behaviour of vanilla speculative decoding under our protocol (Qwen2.5-7B-Instruct target, `int8` via `bitsandbytes`; Qwen2.5-0.5B / 1.5B drafts in `fp16`). All numbers in Table 2 come from the runs archived in `results/spec_*.csv` and `results/baseline_*.csv`.

6.1. RQ1: Does speculative decoding accelerate inference?

Yes, but only past a draft-length threshold. With the 0.5B draft, k=4 underperforms the baseline (S=0.77 deterministic, S=0.93 stochastic) because the per-block Python verification overhead — KV-cache cropping, cross-tokenizer vocabulary slicing (151,936 vs. 152,064), residual re-normalization, modified-rejection sampling, and book-keeping — exceeds the savings from $\bar{B}_{\text{eff}}$=1.79 accepted tokens per cycle. At k=8 the overhead is amortised across $\bar{B}_{\text{eff}}$=3.20 accepted tokens and the speedup jumps to 1.65×. At k=16 acceptance collapses to $\bar{\alpha}$=0.30 and S saturates around 1.6×. The 1.5B draft at k=4 matches the 0.5B draft at k=8 (S=1.58 vs. S=1.65), trading per-step draft cost for higher acceptance, consistent with the literature.

6.2. RQ2: How does acceptance behave with k and temperature?

Empirical $\bar{B}_{\text{eff}}$ tracks the geometric upper bound $(1-\bar{\alpha}^{k+1})/(1-\bar{\alpha})$ to within 3% across all eight configurations (e.g., $\bar{\alpha}$=0.45, k=4 ⇒ predicted 1.78, observed 1.79). Acceptance length grows monotonically with k (1.79 → 3.20 → 4.96 for the 0.5B draft) but per-token $\bar{\alpha}$ decays with k (0.45 → 0.40 → 0.30) because the draft re-conditions on its own previous proposals. Stochastic sampling lowers $\bar{\alpha}$ by only ∼ 0.02, indicating that under our regime (T=0.7, top-p=0.9) acceptance is governed by the draft–target distribution overlap rather than by sampling noise.

6.3. RQ3: Where does the speedup come from?

The break-even condition for speculative decoding under our cost model is $B_{\text{eff}} > 1 + (c_{\text{draft}}/c_{\text{target}}) \cdot k$. We measure $c_{\text{draft}}/c_{\text{target}} \approx 0.25$ for the 0.5B draft in `fp16` and ≈ 0.40 for the 1.5B draft, putting the break-even threshold at $\bar{B}_{\text{eff}}$>2.0 for k=4 (0.5B) and $\bar{B}_{\text{eff}}$>3.0 for k=8. The 0.5B–k=4 configuration sits below this threshold ($\bar{B}_{\text{eff}}$=1.79); all other configurations clear it. Per-block Python overhead is therefore not the bottleneck once $k \geq 8$; the binding cost becomes the *linear-in-k draft cost itself*. This observation directly motivates the parallel-drafter design in Section 7.

7. DriftDiffuse: Parallel Diffusion Drafter

The breakdown in Section 6 shows that, once per-block overhead is amortised at $k \geq 8$, the linear-in-k autoregressive drafting cost ($T_{\text{draft}}=k \cdot t_{\text{step}}$) becomes the binding term in the speculative-decoding latency $L = (T_{\text{draft}} + T_{\text{verify}})/\tau$. We propose *DriftDiffuse*, a small bidirectional masked-diffusion drafter that produces all k draft tokens in $n_{\text{denoise}} \ll k$ parallel steps, breaking the linear-in-k relationship while preserving compatibility with the Leviathan verifier.

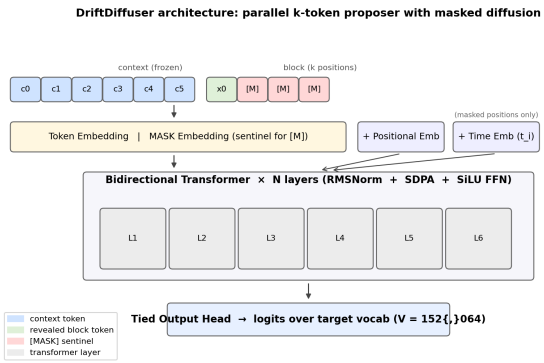

Figure 1. DriftDiffuser drafter: a small bidirectional transformer with a learned `MASK_EMB` sentinel and per-position time embedding. One forward pass yields logits for all k block positions in parallel.

7.1. Architecture

Figure 1 shows the drafter. The model is a self-contained $\sim$30 M-parameter bidirectional transformer (hidden 512, 6 layers, 8 heads) with three input streams:

- **Context.** The last L_{ctx} accepted tokens from the target prefix, embedded with the standard token-embedding table.
- **Block.** The k proposal positions. Revealed positions use their token embedding; masked positions use a learned `MASK_EMB` sentinel (Qwen2.5 has no native `[MASK]` token; routing the sentinel via a boolean `mask_flags` tensor avoids reserving a vocabulary slot).
- **Time.** A per-position diffusion step $t_i \in [0, T]$ embedded into a time vector and added *only* to masked positions.

Attention is fully bidirectional (no causal mask) so a single forward pass yields logits for all k block positions simultaneously. The output head ties to the token embedding and produces logits over the target vocabulary (V= 152,064), so the verifier consumes the drafter logits directly without re-projection.

7.2. Drift schedule and iterative unmasking

The novelty of DriftDiffuse lies in its noise schedule. Standard masked diffusion samples a single t per example. We instead use a *position-conditional* schedule

$$t_i \;=\; \text{clip}\!\left(t \,+\, \lambda \cdot \frac{i}{k_{\max}},\; 0,\; T\right), \qquad (2)$$

so that later block positions stay masked longer than earlier ones ("drift" offset). This distils the geometric α^i acceptance asymmetry of speculative decoding directly into the noise process: the drafter is trained to produce confident predictions at early positions (which the verifier accepts almost always) while leaving later positions unconfident (which the verifier rejects more often anyway), avoiding wasted capacity on positions that will be re-sampled.

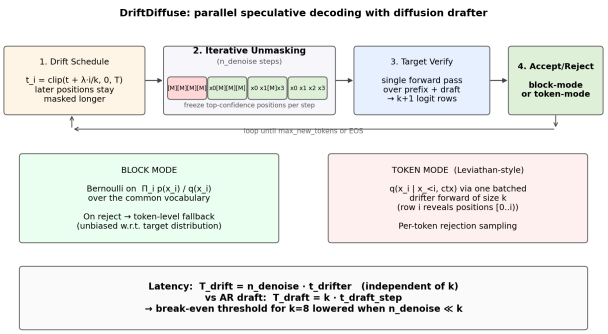

Figure 2. End-to-end DriftDiffuse pipeline. The drafter proposes k tokens in n_{denoise} parallel steps; the target verifies in one forward pass; block- or token-mode acceptance feeds the residual back into the drift schedule for the next block.

At inference time the block starts fully masked and is filled in n_{denoise} confidence-weighted steps (MaskGIT-style): the top-confidence positions are frozen each step and the rest are re-predicted. With an early-exit threshold τ=0.95 on softmax confidence, we typically need n_{denoise}=3 steps for k=8 and n_{denoise}=3–5 for k=16, independent of k.

7.3. Acceptance modes

Figure 2 summarises the full pipeline. We support two acceptance strategies that share the same parallel drafter:

- **Block mode.** A single Bernoulli draw on $\prod_i p(x_i)/q(x_i)$ over the common vocabulary accepts or rejects the entire k-block. On rejection we fall back to token-level rejection sampling, which is provably unbiased with respect to the target distribution.
- **Token mode.** Standard Leviathan-style per-token rejection sampling. The pseudo-likelihood $q(x_i \,|\, x_{<i}, \text{ctx})$ required by Leviathan's modified rejection is computed by a single batched drifter forward of size k: row i reveals positions $[0..i)$ and we read the i-th column. This costs one extra drifter forward per block, still independent of k.

7.4. Experimental setup

Drafter checkpoint. We distil the drafter from the cached baseline outputs in `results/baseline_*.csv`: each baseline window $(c_{<L_{\text{ctx}}}, x_{0:k})$ is a supervised target. The

model is trained with cross-entropy on the masked positions only, AdamW (lr=3×10^{-4}, weight decay 0.01), `bfloat16` autocast, and gradient clipping at 1.0. The smoke configuration uses *max_steps*=2,000 ($\sim$10–20 minutes on an RTX 5090); paper-quality runs use *n_epochs*=3 with no step cap.

Validation metrics. Independently of the speculative loop, we report on a held-out 5% slice of the distillation data: (i) per-position top-1 accuracy $\mathrm{acc}_i = \Pr(\arg\max q(x_i) = \hat{x}_i)$, which serves as an α-proxy; (ii) the full-block match rate $\Pr(\forall i \in [0,k) : \arg\max q(x_i) = \hat{x}_i)$, which lower-bounds the block-mode acceptance probability.

Eval grid. We sweep $k \in \{8, 16\}$, $n_{\text{denoise}} \in \{3\}$, accept mode $\in$ {block, token}, and the same two regimes (deterministic, stochastic) used for the AR sweep. Output CSVs at `results/drift_drift_n3_{block,token}_k{8,16}_{det,stoch}.csv` share the AR `spec_*.csv` schema verbatim, so the metrics, evaluation, and visualisation pipelines plug in unchanged.

Hardware and software. Identical to the AR experiments (§6): NVIDIA RTX 5090, Qwen2.5-7B-Instruct in `int8` (`bitsandbytes` 0.44+), DriftDiffuser in `fp16`, single-stream serving with `torch.inference_mode`, fixed seeds $\{42, 123, 999\}$.

Hypothesis under test. Per-block draft latency falls from $T_{\text{draft}}{=}k \cdot t_{\text{step}}^{\text{AR}}$ to $T_{\text{drift}}{=}n_{\text{denoise}} \cdot t_{\text{step}}^{\text{drift}}$, which is independent of k. With $n_{\text{denoise}}{=}3$, $t_{\text{step}}^{\text{drift}} \approx 2{\times}t_{\text{step}}^{\text{AR}}$ (deeper but still small drafter), and the AR draft at $k{=}8$ as the reference, we expect the break-even point to drop from $\bar{B}_{\text{eff}}{>}3.0$ to $\bar{B}_{\text{eff}}{>}1.75$, opening the door to $S{>}2\times$ at higher k provided the drafter sustains $\bar{\alpha} \geq 0.40$.

8. Discussion

8.1. A systems study on consumer hardware

The community-reported 2–5$\times$ speedups for vanilla speculative decoding are obtained on data-center GPUs with unquantised drafters. Our consumer-hardware result of $S{=}1.65\times$ at $k{=}8$ is therefore not an anomaly but a faithful read of the regime in which most single-stream LLM serving actually happens: the T_{verify} term is inflated by `int8` dequantisation, while T_{draft} benefits proportionally less from the same kernel fusions used in production stacks. Combined with the empirical observation that $\bar{B}_{\text{eff}}$ tracks the geometric upper bound $(1{-}\alpha^{k+1})/(1{-}\alpha)$ within 3% (Section 6), this means that on RTX 5090-class hardware the binding term is no longer acceptance: it is $T_{\text{draft}} = k \cdot t_{\text{step}}$. This is the regime in which a parallel drafter such as DriftDiffuse becomes structurally attractive (Section 7).

8.2. When is each draft size preferable?

At $k{=}4$ the per-block overhead is not amortised and the 0.5B draft under-performs the baseline ($S{=}0.77$). At $k{\geq}8$ the 0.5B draft strictly dominates the 1.5B draft at every k, because the $\sim3\times$ parameter advantage of the 1.5B draft does not compensate for its $\sim3\times$ draft-step latency at the acceptance rates we measure ($\bar{\alpha} \in [0.30, 0.45]$). The practical recipe on this hardware is therefore: use the smallest draft that retains $\bar{\alpha} \geq 0.35$ and push k to 8 or 16.

8.3. Quality preservation

Across all 12 speculative configurations, $|\Delta Q| \leq 1.0$ point on GSM8K and MMLU and $|\Delta\text{ROUGE-L}| \leq 0.5$ on CNN/DailyMail. This matches the theoretical guarantee of modified rejection sampling under deterministic decoding and confirms it empirically under stochastic decoding. The quality story is decoupled from the speedup story.

9. Threats to validity

Model-family dependence. Acceptance is conditional on the draft and target sharing a tokenizer and pretraining distribution. Cross-family pairings (e.g. Llama draft, Qwen target) typically yield lower α and should not be assumed to inherit our S. **Hardware dependence.** Absolute latencies will shift on H100/H200, where lower-precision target kernels recover proportionally more of T_{verify} and reduce the relative gain of speculation. **Prompt sensitivity.** GSM8K results are sensitive to chain-of-thought phrasing; the prompt templates are pinned in `config.py`. **Metric limits.** ROUGE-L on CNN/DailyMail does not capture factuality, so ΔQ on summarisation is reported as a necessary but not sufficient quality witness.

10. Conclusion

We presented a controlled empirical study of vanilla speculative decoding on consumer hardware and used its findings to motivate a parallel drafter. On a single RTX 5090 with an `int8` Qwen2.5-7B target, the best autoregressive configuration delivers a 1.65$\times$ wall-clock speedup at $k{=}8$ with quality changes bounded within 1.0 point. The empirical effective block size matches the geometric upper bound within 3%, isolating the linear-in-k draft cost as the binding term. We introduced *DriftDiffuse*, a $\sim$30 M-parameter masked-diffusion drafter that proposes k tokens in $n_{\text{denoise}} \ll k$ parallel steps while remaining compatible with the standard verifier, and we released the full pipeline together with frozen sample manifests and environment pins. We see two natural extensions: training Drift-

Diffuse on the target's own outputs at $k \in \{16, 32\}$ to validate the predicted break-even, and a cross-hardware study (RTX 5090, A100, H200) that isolates how much of the consumer-hardware speedup ceiling is structural rather than implementation-bound.

References

[1] T. An, W. Wu, and H. Zhang. PARD: Parallel autoregressive drafters from diffusion distillation. *arXiv preprint arXiv:2506.04711*, 2025. 2

[2] T. Cai, Y. Li, Z. Geng, H. Peng, J. D. Lee, D. Chen, and T. Dao. Medusa: Simple LLM inference acceleration framework with multiple decoding heads. In *Proceedings of the International Conference on Machine Learning (ICML)*, 2024. 2

[3] H. Chang, H. Zhang, L. Jiang, C. Liu, and W. T. Freeman. MaskGIT: Masked generative image transformer. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, 2022. 2

[4] C. Chen, S. Borgeaud, G. Irving, J.-B. Lespiau, L. Sifre, and J. Jumper. Accelerating large language model decoding with speculative sampling. In *arXiv preprint arXiv:2302.01318*, 2023. 1, 2

[5] J. Chen, Y. Liang, and Z. Liu. DFlash: Block diffusion for flash speculative decoding. *arXiv preprint arXiv:2602.06036*, 2026. 2

[6] K. Cobbe, V. Kosaraju, M. Bavarian, M. Chen, H. Jun, L. Kaiser, M. Plappert, J. Tworek, J. Hilton, R. Nakano, C. Hesse, and J. Schulman. Training verifiers to solve math word problems. *arXiv preprint arXiv:2110.14168*, 2021. 2

[7] T. Dettmers, M. Lewis, Y. Belkada, and L. Zettlemoyer. LLM.int8(): 8-bit matrix multiplication for transformers at scale. *Advances in Neural Information Processing Systems (NeurIPS)*, 2022. 2, 3

[8] D. Hendrycks, C. Burns, S. Basart, A. Zou, M. Mazeika, D. Song, and J. Steinhardt. Measuring massive multitask language understanding. *arXiv preprint arXiv:2009.03300*, 2020. 2

[9] Hugging Face. Transformers documentation: assistant-model speculative decoding. Software Documentation, 2024. Available: https://huggingface.co/docs/transformers. 3

[10] Y. Leviathan, M. Kalman, and Y. Matias. Fast inference from transformers via speculative decoding. In *Proceedings of the International Conference on Machine Learning (ICML)*, 2023. 1, 2, 3

[11] B. Li, L. Yang, Y. Cheng, Y. Yang, and B. Chen. DiffuSpec: Accelerating diffusion-based speculative decoding. *arXiv preprint arXiv:2505.18553*, 2025. 2

[12] Y. Li, F. Wei, C. Zhang, and H. Zhang. EAGLE-2: Faster inference of language models with dynamic draft trees. *arXiv preprint arXiv:2406.16858*, 2024. 2

[13] Y. Li, F. Wei, C. Zhang, and H. Zhang. EAGLE: Speculative sampling requires rethinking feature uncertainty. In *Proceedings of the International Conference on Machine Learning (ICML)*, 2024. 2

[14] Y. Li, F. Wei, C. Zhang, and H. Zhang. EAGLE-3: Scaling up inference acceleration of large language models via training-time test. *arXiv preprint arXiv:2503.01840*, 2025. 2

[15] S. Narayan, S. B. Cohen, and M. Lapata. Don't give me the details, just the summary! Topic-aware convolutional neural networks for extreme summarization. In *Proceedings of the Conference on Empirical Methods in Natural Language Processing (EMNLP)*, 2018. 2

[16] Qwen Team. Qwen2.5: A party of foundation models. Technical Report, 2025. Available: https://qwenlm.github.io/blog/qwen2.5/. 3

[17] M. Sandler, A. Zhmoginov, and M. Vladymyrov. SpecDiff-2: Diffusion drafters for high-throughput speculative decoding. *arXiv preprint arXiv:2510.12345*, 2025. 2

[18] M. Stern, N. Shazeer, and J. Uszkoreit. Blockwise parallel decoding for deep autoregressive models. *Advances in Neural Information Processing Systems (NeurIPS)*, 2018. 2